

BEACONHOUSE NATIONAL UNIVERSITY



LAB 7 | WEEK 7

Name: Saad Mughal

BNU ID: F2023-009

Due Date: 29 Mar, 2025

Lab Manual

For

ADMS(SEC_B)

Course Instructor	Ms. Talia Noreen
Lab Instructor	Mr. Ahsan Atiq
Semester	Spring 2025

1. Find customers (Name, City) who have placed an order where the amount is higher than the average order amount of customers from the same city, but only if their total spending is also above the city-wide average.

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to find customers (Name, City) who have placed an order where the amount is higher than the average order amount of customers from the same city, but only if their total spending is also above the city-wide average.

```

-- 1. Find customers (Name, City) who have placed an order where the amount is higher than the average order amount of customers from the same city, but only if their total spending is also above the city-wide average.
45 SELECT c.Name, c.City
46 FROM Customers c
47 JOIN Orders o ON c.CustomerID = o.CustomerID
48 WHERE o.Amount > (
49     SELECT AVG(o2.Amount)
50     FROM Orders o2
51     JOIN Customers c2 ON o2.CustomerID = c2.CustomerID
52     WHERE c2.City = c.City
53 )
54 AND (
55     SELECT SUM(o3.Amount)
56     FROM Orders o3
57     WHERE o3.CustomerID = c.CustomerID
58 ) > (
59     SELECT AVG(total_spent)
60     FROM (
61         SELECT c2.City, SUM(o2.Amount) AS total_spent
62         FROM Orders o2
63         JOIN Customers c2 ON o2.CustomerID = c2.CustomerID
64         WHERE c2.City = c.City
65         GROUP BY c2.CustomerID
66     ) AS city_avg
67 );

```

The Results window shows the output of the query, displaying two rows of data:

Name	City
Alice	New York

The Output window shows the execution of the query, including messages and error codes.

2. List products (ProductName, Price) that have been ordered by at least two distinct customers, ensuring that the product price is greater than the highest-priced product ordered by CustomerID 1.

The screenshot shows the MySQL Workbench interface. The SQL editor contains a query to list products (ProductName, Price) that have been ordered by at least two distinct customers, ensuring that the product price is greater than the highest-priced product ordered by CustomerID 1.

```

-- 2. List products (ProductName, Price) that have been ordered by at least two distinct customers, ensuring that the product price is greater than the highest-priced product ordered by CustomerID 1.
69 SELECT p.ProductName, p.Price
70 FROM Products p
71 WHERE p.Price > COALESCE(
72     SELECT MAX(p2.Price)
73     FROM Products p2
74     JOIN Orders o2 ON p2.ProductID = o2.ProductID
75     WHERE o2.CustomerID = 1
76 )
77 AND (
78     SELECT COUNT(DISTINCT o3.CustomerID)
79     FROM Orders o3
80     GROUP BY o3.ProductID
81     HAVING COUNT(DISTINCT o3.CustomerID) >= 2
82 )
83 );

```

The Results window shows the output of the query, displaying two rows of data:

ProductName	Price
Product 1	100
Product 2	200

The Output window shows the execution of the query, including messages and error codes.

3. Find customers (Name, City) who placed an order where the amount is higher than the highest single order amount of any customer from the same city, but only if they have placed at least two orders.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

79 SELECT o3.ProductID
80 FROM Orders o3
81 GROUP BY o3.ProductID
82 HAVING COUNT(DISTINCT o3.CustomerID) >= 2
83
84 -- 3. Find customers (Name, City) who placed an order where the amount is higher than the highest single order amount of any customer from the same city, but only if they have placed at least two orders.
85 SELECT c.Name, c.City
86 FROM Customers c
87 JOIN Orders o ON c.CustomerID = o.CustomerID
88 WHERE o.Amount > (
89     SELECT MAX(o2.Amount)
90     FROM Orders o2
91     JOIN Customers c2 ON o2.CustomerID = c2.CustomerID
92     WHERE c2.City = c.City AND c2.CustomerID <> c.CustomerID
93 )
94 AND (
95     SELECT COUNT(*)
96     FROM Orders o3
97     WHERE o3.CustomerID = c.CustomerID
98 ) >= 2;
99
100
101 -- 4. Show customers (Name, City) whose total spending is greater than the total spending of every other customer from their city, ensuring they have at least three distinct orders.
102 SELECT c.Name, c.City

```

The Results tab shows the output of the query, displaying columns Name and City. The Output tab shows the execution progress and messages.

4. Show customers (Name, City) whose total spending is greater than the total spending of every other customer from their city, ensuring they have at least three distinct orders.

The screenshot shows the MySQL Workbench interface. The SQL editor contains the following query:

```

92 JOIN Customers c2 ON o2.CustomerID = c2.CustomerID
93 WHERE c2.City = c.City AND c2.CustomerID <> c.CustomerID
94
95 AND (
96     SELECT COUNT(*)
97     FROM Orders o3
98     WHERE o3.CustomerID = c.CustomerID
99 ) >= 2;
100
101 -- 4. Show customers (Name, City) whose total spending is greater than the total spending of every other customer from their city, ensuring they have at least three distinct orders.
102 SELECT c.Name, c.City
103 FROM Customers c
104 JOIN Orders o ON c.CustomerID = o.CustomerID
105 GROUP BY c.CustomerID, c.Name, c.City
106 HAVING SUM(o.Amount) > ALL (
107     SELECT SUM(o2.Amount)
108     FROM Orders o2
109     JOIN Customers c2 ON o2.CustomerID = c2.CustomerID
110     WHERE c2.City = c.City AND c2.CustomerID <> c.CustomerID
111     GROUP BY c2.CustomerID
112 )
113 AND COUNT(DISTINCT o.OrderID) >= 3;
114
115 -- 5. INSERT new customers into a backup table if they have placed an order in every region where at least one other customer from their city has ordered.

```

The Results tab shows the output of the query, displaying columns Name and City. The Output tab shows the execution progress and messages.

5. INSERT new customers into a backup table if they have placed an order in every region where at least one other customer from their city has ordered.

```

109 JOIN Customers c2 ON o2.CustomerID = c2.CustomerID
110 WHERE c2.City = c.City AND c2.CustomerID <> c.CustomerID
111 GROUP BY c2.CustomerID
112 )
113 AND COUNT(DISTINCT o.OrderID) >= 3;
114
115 -- 5. INSERT new customers into a backup table if they have placed an order in every region where at least one other customer from their city has ordered.
116 CREATE TABLE Customer_Backup (
117   CustomerID INT PRIMARY KEY,
118   Name VARCHAR(255),
119   City VARCHAR(255)
120 );
121 INSERT INTO Customer_Backup (CustomerID, Name, City)
122 SELECT DISTINCT c.CustomerID, c.Name, c.City
123 FROM Customers c
124 WHERE NOT EXISTS (
125   SELECT 1 FROM (
126     SELECT DISTINCT o.Region FROM Orders o
127     WHERE o.CustomerID = c.CustomerID
128   ) AS CustomerRegions
129   WHERE CustomerRegions.Region NOT IN (
130     SELECT DISTINCT o2.Region FROM Orders o2
131     JOIN Customers c2 ON o2.CustomerID = c2.CustomerID
132     WHERE c2.City = c.City
133   )
134 );
135
136 -- 6. UPDATE the price of all products that have been ordered by at least two distinct customers, increasing their price by 15%, but only if they have also been ordered in at least two different regions.
137 SET SQL_SAFE_UPDATES = 0;
138
139 UPDATE Products p
140 JOIN (

```

#	Time	Action	Message	Duration / Fetch
39	21:58:23	DROP TEMPORARY TABLE TempMeAmount	0 rows(s) affected	0.000 sec
40	22:01:58	INSERT INTO Customer_Backup (CustomerID, Name, City) SELECT DISTINCT c.CustomerID, c.Name, c.City FROM Customers c WHERE NOT EXISTS (SELECT 1 FROM (SELECT DISTINCT o2.Region FROM Orders o2 JOIN Customers c2 ON o2.CustomerID = c2.CustomerID WHERE c2.City = c.City EXCEPT SELECT DISTINCT o3.Region FROM Orders o3 WHERE o3.CustomerID = c.CustomerID) AS MissingRegions)	Error Code: 1062 Duplicate entry '1' for key 'customerbackup.PRIMARY'	0.000 sec
41	22:02:06	CREATE TABLE Customer_Backup (CustomerID INT PRIMARY KEY, Name VARCHAR(255), City VARCHAR(255))	0 rows(s) affected	0.015 sec
42	22:02:09	INSERT INTO Customer_Backup (CustomerID, Name, City) SELECT DISTINCT c.CustomerID, c.Name, c.City FROM Customers c WHERE NOT EXISTS (SELECT 1 FROM (SELECT DISTINCT o2.Region FROM Orders o2 JOIN Customers c2 ON o2.CustomerID = c2.CustomerID WHERE c2.City = c.City EXCEPT SELECT DISTINCT o3.Region FROM Orders o3 WHERE o3.CustomerID = c.CustomerID) AS MissingRegions)	4 rows(s) affected Records: 4 Duplicates: 0 Warnings: 0	0.000 sec

6. UPDATE the price of all products that have been ordered by at least two distinct customers, increasing their price by 15%, but only if they have also been ordered in at least two different regions.

```

125 WHERE NOT EXISTS (
126   SELECT 1
127   FROM (
128     SELECT DISTINCT o2.Region
129     FROM Orders o2
130     JOIN Customers c2 ON o2.CustomerID = c2.CustomerID
131     WHERE c2.City = c.City
132   ) EXCEPT
133   SELECT DISTINCT o3.Region
134   FROM Orders o3
135   WHERE o3.CustomerID = c.CustomerID
136 ) AS MissingRegions
137 );
138
139 -- 6. UPDATE the price of all products that have been ordered by at least two distinct customers, increasing their price by 15%, but only if they have also been ordered in at least two different regions.
140 SET SQL_SAFE_UPDATES = 0;
141
142 UPDATE Products p
143 JOIN (
144   SELECT ProductID
145   FROM Orders
146   GROUP BY ProductID
147   HAVING COUNT(DISTINCT CustomerID) >= 2
148   AND COUNT(DISTINCT Region) >= 2
149 ) o ON p.ProductID = o.ProductID
150 SET p.Price = p.Price * 1.15;
151
152 SET SQL_SAFE_UPDATES = 1;
153
154 -- 7. DELETE all orders where the order amount is lower than the minimum order amount placed in the same region by a customer who has at least two orders in that region.
155 DELETE FROM Orders

```

#	Time	Action	Message	Duration / Fetch
29	21:54:36	INSERT INTO Customer_Backup (CustomerID, Name, City) SELECT c.CustomerID, c.Name, c.City FROM Customers c WHERE NOT EXISTS (SELECT 1 FROM (SELECT DISTINCT o2.Region FROM Orders o2 JOIN Customers c2 ON o2.CustomerID = c2.CustomerID WHERE c2.City = c.City EXCEPT SELECT DISTINCT o3.Region FROM Orders o3 WHERE o3.CustomerID = c.CustomerID) AS MissingRegions)	Error Code: 1062 Duplicate entry '1' for key 'customerbackup.PRIMARY'	0.000 sec
30	21:54:39	SET SQL_SAFE_UPDATES = 0	0 rows(s) affected	0.000 sec
31	21:55:03	UPDATE Products p JOIN (SELECT ProductID FROM Orders GROUP BY ProductID HAVING COUNT(DISTINCT CustomerID) >= 2 AND COUNT(DISTINCT Region) >= 2) o ON p.ProductID = o.ProductID SET p.Price = p.Price * 1.15;	1 rows(s) affected Rows matched: 1 Changed: 1 Warnings: 0	0.000 sec
32	21:55:06	SET SQL_SAFE_UPDATES = 1	0 rows(s) affected	0.000 sec

7. DELETE all orders where the order amount is lower than the minimum order amount placed in the same region by a customer who has at least two orders in that region.

```

147 SET p.Price = p.Price * 1.15;
148
149 SET SQL_SAFE_UPDATES = 1;
150
151
152 -- 7. DELETE all orders where the order amount is lower than the minimum order amount placed in the same region by a customer who has at least two orders in that region.
153 CREATE TEMPORARY TABLE TempMinOrders AS
154 SELECT o2.Region, MIN(o2.Amount) AS MinOrderAmount
155 FROM Orders o2
156 WHERE o2.CustomerID IN (
157     SELECT CustomerID
158     FROM Orders
159     GROUP BY CustomerID, Region
160     HAVING COUNT(*) >= 2
161 )
162 GROUP BY o2.Region;
163
164 DELETE FROM Orders
165 WHERE Amount < (
166     SELECT MinOrderAmount
167     FROM TempMinOrders
168     WHERE Orders.Region = TempMinOrders.Region
169 );
170
171 DROP TEMPORARY TABLE TempMinOrders;
172
173 -- 8. INSERT products into a special discount table if they have been ordered by at least two customers from different cities and their price is below the average price of products ordered in the same region.
174 CREATE TABLE IF NOT EXISTS DiscountProducts (
175     ProductID INT PRIMARY KEY,
176     ProductName VARCHAR(255),
177     Price INT
178 );

```

Table: customerbackup

Columns: CustomerID (INT PK), Name (VARCHAR(255)), City (VARCHAR(255))

Output

#	Time	Action	Message	Duration / Fetch
43	22:02:38	DELETE FROM Orders WHERE Amount < (SELECT MinOrderAmount FROM (--Get the minimum order amount for customers with at least two orders in the same region	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE that uses a KEY column. To disable safe mode, to...	0.015 sec
44	22:05:03	CREATE TEMPORARY TABLE TempMinOrders AS SELECT o2.Region, MIN(o2.Amount) AS MinOrderAmount FROM Orders o2 WHERE o2.CustomerID IN (SELECT CustomerID FROM Orders GROUP BY CustomerID, Region HAVING COUNT(*) >= 2) GROUP BY o2.Region;	1 row(s) affected Records: 1 Duplicates: 0 Warnings: 0	0.000 sec
45	22:05:05	DELETE FROM Orders WHERE Amount < (SELECT MinOrderAmount FROM TempMinOrders WHERE Orders.Region = TempMinOrders.Region);	Error Code: 1175. You are using safe update mode and you tried to update a table without a WHERE that uses a KEY column. To disable safe mode, to...	0.000 sec
46	22:05:09	DROP TEMPORARY TABLE TempMinOrders;	0 row(s) affected	0.000 sec

8. INSERT products into a special discount table if they have been ordered by at least two customers from different cities and their price is below the average price of products ordered in the same region.

```

163 FROM Orders
164 WHERE Region = o2.Region
165 GROUP BY CustomerID
166 HAVING COUNT(*) >= 2
167 )
168 );
169
170 -- 8. INSERT products into a special discount table if they have been ordered by at least two customers from different cities and their price is below the average price of products ordered in the same region.
171 CREATE TABLE IF NOT EXISTS DiscountProducts (
172     ProductID INT PRIMARY KEY,
173     ProductName VARCHAR(255),
174     Price INT
175 );
176
177 INSERT INTO DiscountProducts (ProductID, ProductName, Price)
178 SELECT DISTINCT p.ProductID, p.ProductName, p.Price
179 FROM Products p
180 JOIN Orders o ON p.ProductID = o.ProductID
181 WHERE p.ProductID IN (
182     SELECT o2.ProductID
183     FROM Orders o2
184     JOIN Customers c2 ON o2.CustomerID = c2.CustomerID
185     GROUP BY o2.ProductID
186     HAVING COUNT(DISTINCT c2.City) >= 2
187 )
188 AND p.Price < (
189     SELECT AVG(p2.Price)
190     FROM Products p2
191     JOIN Orders o3 ON p2.ProductID = o3.ProductID
192     WHERE o3.Region = o.Region
193 );
194
195 SET SQL_SAFE_UPDATES = 1;
196
197 DELETE FROM Orders WHERE Amount < ( SELECT MIN(o2.Amount) FROM Orders o2 WHERE o2.Region = Orders.Region AND o2.CustomerID IN ( SELECT CustomerID FROM Orders GROUP BY CustomerID, Region HAVING COUNT(*) >= 2 ) )
198
199 CREATE TABLE IF NOT EXISTS DiscountProducts ( ProductID INT PRIMARY KEY, ProductName VARCHAR(255), Price INT )
200
201 INSERT INTO DiscountProducts (ProductID, ProductName, Price) SELECT DISTINCT p.ProductID, p.ProductName, p.Price FROM Products p JOIN Orders o ON p.ProductID = o.ProductID WHERE o.Region = o2.Region AND o2.CustomerID IN ( SELECT CustomerID FROM Orders GROUP BY CustomerID, Region HAVING COUNT(*) >= 2 ) AND p.Price < ( SELECT AVG(p2.Price) FROM Products p2 JOIN Orders o3 ON p2.ProductID = o3.ProductID WHERE o3.Region = o.Region )

```

Table: customerbackup

Columns: CustomerID (INT PK), Name (VARCHAR(255)), City (VARCHAR(255))

Output

#	Time	Action	Message	Duration / Fetch
32	21:55:06	SET SQL_SAFE_UPDATES = 1;	0 row(s) affected	0.000 sec
33	21:55:28	DELETE FROM Orders WHERE Amount < (SELECT MIN(o2.Amount) FROM Orders o2 WHERE o2.Region = Orders.Region AND o2.CustomerID IN (SELECT CustomerID FROM Orders GROUP BY CustomerID, Region HAVING COUNT(*) >= 2))	Error Code: 1053. You can't specify target table 'Orders' for update in FROM clause	0.000 sec
34	21:55:40	CREATE TABLE IF NOT EXISTS DiscountProducts (ProductID INT PRIMARY KEY, ProductName VARCHAR(255), Price INT)	0 row(s) affected	0.001 sec
35	21:55:43	INSERT INTO DiscountProducts (ProductID, ProductName, Price) SELECT DISTINCT p.ProductID, p.ProductName, p.Price FROM Products p JOIN Orders o ON p.ProductID = o.ProductID WHERE o.Region = o2.Region AND o2.CustomerID IN (SELECT CustomerID FROM Orders GROUP BY CustomerID, Region HAVING COUNT(*) >= 2) AND p.Price < (SELECT AVG(p2.Price) FROM Products p2 JOIN Orders o3 ON p2.ProductID = o3.ProductID WHERE o3.Region = o.Region)	1 row(s) affected Records: 1 Duplicates: 0 Warnings: 0	0.000 sec